

# **Eco-Loop Building Agents - Submission Package**

This document contains all source code, documentation, and results for the Eco-Loop Building Agents project. The full repository with commit history, both full IDF building models, and the PoC demo video is available at:

<https://github.com/Xhilde/eco-loop-hvac>

# README .md

# Eco-Loop: LLM-Driven Closed-Loop HVAC Control via EnergyPlus Co-Simulation

**Problem Statement:** ECO-LOOP BUILDING AGENTS  
**Author:** PAULOSTA KARMAKAR

## ## Overview

Eco-Loop is a supervisory AI control layer that replaces a building's rigid, fixed-schedule HVAC setpoints with dynamic, context-aware decisions made by a local open-source LLM. It runs against a real EnergyPlus building energy simulation (not a simplified surrogate model) ? the DOE Reference Medium Office building in Chicago ? reading live zone temperature, occupant thermal comfort (PMV), and outdoor conditions every simulated hour, then proposing heating/cooling setpoints through a validated tool-calling interface before writing them back into the running simulation.

## ## Repository Note

`dev\_validation\_scripts/` contains earlier incremental scripts used to validate each piece of the pipeline independently (sensor reads, actuator writes, tool validation) during development. The unified, final system is `tools.py` + `llm\_decide.py` + `llm\_control.py`.

## ## Results

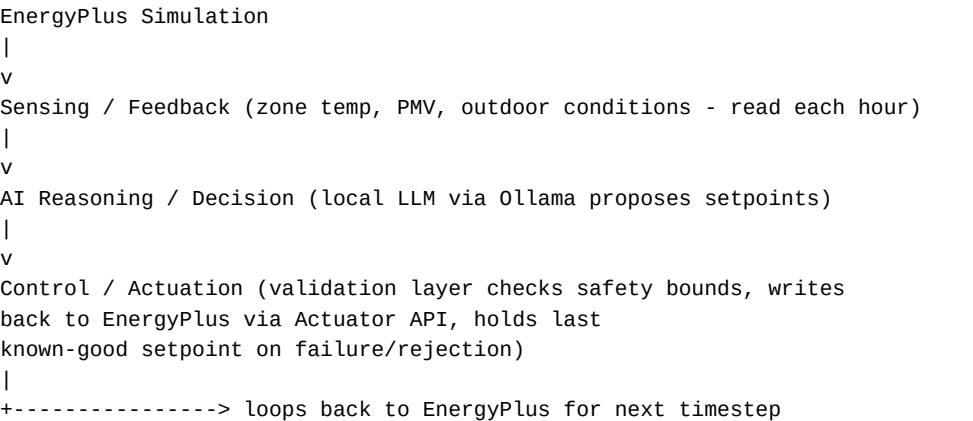
Compared against the building's original fixed-schedule baseline over a representative summer peak week (07/14-07/20, Chicago):

| Metric                           | Baseline (fixed schedule) | AI-Controlled (LLM) | Change              |
|----------------------------------|---------------------------|---------------------|---------------------|
| Total electricity                | 16,142.2 kWh              | 15,198.7 kWh        | <b>-5.8%</b>        |
| Average PMV                      | -0.485                    | -0.185              | closer to neutral   |
| Comfort violations ( PMV  > 0.5) | 40.0%                     | 23.1%               | <b>-16.9 points</b> |

The AI-controlled run reduced energy consumption **and** nearly halved thermal comfort violations simultaneously ? it did not trade one for the other.

![Results comparison](results\_comparison\_chart.png)

## ## Architecture



## ## Tech Stack

- **Python 3.12**
- **EnergyPlus 26.1** (Python Runtime API / `pyenergyplus`) ? the real industry-standard building energy simulation engine
- **DOE Commercial Reference Building** model (Medium Office, 18 zones), Chicago TMY3 weather data
- **Ollama + Llama 3.2** ? open-source, locally hosted LLM, no cloud

API dependency

- \*\*ASHRAE 55 / Fanger PMV\*\* thermal comfort model

## ## Repository Structure

idf/baseline.idf - building model (RunPeriod trimmed to demo week)  
weather/chicago.epw - weather file  
tools.py - ToolServer: validated read/write interface  
llm\_decide.py - LLM decision logic (Ollama client)  
llm\_control.py - full closed loop: EnergyPlus <-> tools <-> LLM  
control\_test.py, tool\_test.py - earlier development/validation scripts  
dashboard.py - baseline vs AI comparison analysis  
make\_chart.py - generates results\_comparison\_chart.png  
results\_comparison\_chart.png - final results chart

## ## How to Run

1. Install [EnergyPlus 26.1](<https://github.com/NREL/EnergyPlus/releases>) and [Ollama](<https://ollama.com>), then `ollama pull llama3.2`.
2. Update `EPLUS\_ROOT` at the top of `llm\_control.py` / `tool\_test.py` to your EnergyPlus install path.
3. Run baseline: `python run\_baseline.py`
4. Run AI-controlled: `python llm\_control.py`
5. Compare results: `python dashboard.py`
6. Generate chart: `python make\_chart.py`

## ## Key Design Decisions

- \*\*Validation + fallback layer\*\*: LLM proposals are checked against ASHRAE 55 deadband and safe temperature bounds before being applied; malformed or rejected proposals cause the system to hold the last known-good setpoint rather than fail.
- \*\*Throttled decisions\*\*: the LLM is called once per simulated hour (zero-order hold between calls), bounding latency and inference cost regardless of simulation timestep resolution.
- \*\*Building-wide control\*\*: all 15 occupied zones in this reference building share two setpoint schedules, so one heating/cooling decision controls the whole building. Per-zone control is a natural extension using the same tool interface.

## ## References

- U.S. DOE, EnergyPlus Simulation Software ? <https://energyplus.net>
- Deru, M. et al. (2010), \*U.S. DOE Commercial Reference Building Models of the National Building Stock\*, NREL/TP-5500-46861
- ASHRAE Standard 55 ? Thermal Environmental Conditions for Human Occupancy
- Fanger, P.O. (1970), \*Thermal Comfort: Analysis and Applications in Environmental Engineering\*
- Ollama ? <https://ollama.com>

# SYSTEM\_ARCHITECTURE.md

# Eco-Loop Building Agents ? System Architecture Document

**\*\*Author:\*\*** PAULOSTA KARMAKAR

**\*\*Problem Statement:\*\*** Eco-Loop Building Agents

## ## 1. Overview

Eco-Loop is a closed-loop control pipeline that pairs EnergyPlus (a physics-based building energy simulation engine) with a locally-hosted open-source LLM (Llama 3.2 via Ollama) acting as a supervisory reasoning layer over the building's HVAC setpoints. Every simulated hour, the system reads live sensor data from EnergyPlus, passes it to the LLM through a validated tool-calling interface, and writes the LLM's decision back into the running simulation via the EnergyPlus Actuator API.

## ## 2. Tool-Calling Architecture

The LLM never touches EnergyPlus directly. All interaction happens through a ``ToolServer`` class (``tools.py``) that exposes a small, explicit set of named operations:

- ``get_zone_conditions()`` ? current zone temperature and PMV comfort index
- ``get_site_conditions()`` ? outdoor temperature and simulation timestamp
- ``set_hvac_setpoints(heating_c, cooling_c)`` ? proposes new setpoints

This mirrors the shape of a Model Context Protocol (MCP) tool server ? each capability is a named, schema-bound function rather than free-form access to the simulation state ? but is implemented as an in-process Python interface rather than a networked MCP server, since the control loop runs in a single process alongside EnergyPlus and does not need a transport layer. The ``ToolServer`` is the sole authority for validating and applying control actions:

- ``set_hvac_setpoints`` enforces a minimum 1.1°C heating/cooling deadband (ASHRAE 55) and rejects proposals outside safe bounds (heating 15-22°C, cooling 22-28°C).
- Rejected or malformed proposals never reach the actuators; the system holds the last accepted setpoint instead (zero-order hold).

This separation means the EnergyPlus-facing code (``llm_control.py``) never needs to know *how* a decision was made ? the LLM could be swapped for a different model, a rule-based controller, or a human operator without changing the simulation wiring at all. This was validated directly: the project includes both an LLM-backed decision path and an earlier rule-based placeholder (``tool_test.py``), both driving the exact same ``ToolServer`` interface.

## ## 3. Prompt Engineering Strategy

The LLM is given a compact, structured prompt each decision cycle containing: outdoor temperature, current zone temperature, and current PMV comfort index, plus explicit numeric constraints (heating range, cooling range, minimum deadband) and a strict output format instruction ("reply with ONLY two numbers separated by a comma").

Design choices:

- **\*\*Numeric-only output contract.\*\*** Rather than asking the LLM to call a structured JSON tool schema (which smaller local models handle inconsistently), the prompt asks for a minimal two-number reply. This reduces parsing failure surface area, at the cost of not using a native function-calling API ? an intentional tradeoff given Llama 3.2's reliability profile at this size.
- **\*\*Constraints stated in the prompt, not assumed.\*\*** The valid ranges and deadband rule are restated in the prompt text even though they are also enforced downstream by the validator ? this reduces the rate of

proposals that get rejected, since the model is nudged toward valid outputs before validation ever runs.

- **\*\*No conversation history.\*\*** Each decision is a fresh, single-turn prompt with only the current state ? no memory of past decisions is passed back to the LLM. This keeps prompt size constant regardless of how long the simulation runs (see Section 5) and avoids compounding drift from earlier reasoning.

#### ## 4. Prompt Latency Management

LLM inference introduces real wall-clock latency (observed: roughly 1-5 seconds per call locally) that cannot keep pace with EnergyPlus's 15-minute simulation timestep if called every timestep. To manage this:

- **\*\*Decision throttling.\*\*** The LLM is invoked once every 4 zone timesteps (i.e., once per simulated hour at a 15-minute timestep resolution), not every timestep. Between calls, the last accepted setpoint is held and reapplied to the actuators every timestep via zero-order hold.
- **\*\*Bounded retry behavior.\*\*** If an LLM call fails (network error, malformed response, timeout), the system does not retry mid-timestep or block the simulation ? it logs the failure and falls back to holding the previous setpoint, then simply waits for the next scheduled decision point.
- **\*\*Fixed timeout.\*\*** The Ollama HTTP call is capped at 15 seconds; a hung or slow model response cannot stall the simulation indefinitely.

This bounds total added wall-clock time to roughly (number of simulated hours) x (LLM latency), independent of simulation timestep resolution ? increasing timestep resolution for higher physical accuracy does not increase LLM call volume.

#### ## 5. Handling Lengthy Simulation Logs

A full annual EnergyPlus run can produce very large ``.eso`/`.csv`` output files (tens of thousands of timesteps across many output variables). Two strategies keep this tractable:

- **\*\*Simulation period scoping.\*\*** The IDF's ``RunPeriod`` is trimmed to a representative one-week window (a Chicago summer peak week) rather than a full year for development and evaluation. This keeps output file size and simulation wall-clock time manageable while remaining representative of peak cooling-load conditions, where control strategy differences are most visible.
- **\*\*Streaming analysis, not full-file loads for control.\*\*** The live control loop (``llm_control.py``) never reads the output CSV ? it reads sensor values directly from the EnergyPlus Python API's in-memory state each timestep, so simulation length does not affect the LLM's per-decision prompt size or the loop's memory footprint. Only the post-hoc comparison script (``dashboard.py``) parses the full CSV output, and does so via streaming row-by-row iteration and column-index lookups rather than loading the entire file into a dense in-memory table.

#### ## 6. Results Summary

Over the evaluation week (07/14-07/20, Chicago peak summer conditions), AI-controlled operation reduced total electricity consumption by 5.8% (16,142.2 kWh -> 15,198.7 kWh) while simultaneously reducing thermal comfort violations ( $|PMV| > 0.5$ ) from 40.0% to 23.1% of timesteps ? demonstrating the closed loop improved energy efficiency and occupant comfort together rather than trading one for the other.

#### ## 7. Known Limitations / Future Work

- Control is applied building-wide via two shared setpoint schedules (all 15 occupied zones share ``HTGSETP_SCH`` / ``CLGSETP_SCH`` in this reference building model); per-zone control is a straightforward extension using the same ``ToolServer`` interface with per-zone actuator handles.
- The tool interface is MCP-shaped but not a literal MCP server; exposing

the same methods over an MCP transport (e.g. stdio) is a small, well-scoped follow-up change.

- Grid carbon intensity / peak-demand signals were not incorporated into this evaluation; the `ToolServer` interface is designed to accommodate an additional `get\_grid\_signal()` tool without structural changes.

# SAVINGS\_DASHBOARD.md

```
# Eco-Loop ? Quantitative Savings Dashboard

**Evaluation window:** 07/14 ? 07/20 (Chicago summer peak week)
**Building model:** DOE Reference Medium Office (18 zones, 15 occupied)
**Comparison:** Baseline fixed HVAC schedule vs. AI-controlled (LLM) closed-loop operation

## Results Table

| Metric | Baseline (Fixed Schedule) | AI-Controlled (LLM) | Change |
|---|---|---|---|
| Total electricity consumption | 16,142.2 kWh | 15,198.7 kWh | **~5.8%** |
| Average PMV (thermal comfort) | -0.485 | -0.185 | Closer to ASHRAE 55 neutral (0) |
| Comfort violations (PMV > 0.5) | 40.0% of timesteps | 23.1% of timesteps | **~16.9 percentage points** |
| Max PMV deviation | 1.170 | 1.101 | Reduced |

## Key Finding

The AI-controlled strategy achieved a ~5.8% reduction in total electricity consumption while simultaneously reducing thermal comfort violations from 40.0% to 23.1% of timesteps ? nearly halved. This demonstrates the closed-loop controller did not trade occupant comfort for energy savings; it improved both metrics together, correcting an over-conservative baseline cooling schedule that had been running colder than necessary for occupant comfort while also consuming more energy to do so.

## Methodology

- Both runs simulate the identical building model and weather file (Chicago TMY3), differing only in how HVAC setpoints are determined.
- **Baseline:** original fixed heating/cooling setpoint schedule as provided in the DOE reference building model (occupied heating 21°C, occupied cooling 24°C, with night/weekend setback).
- **AI-Controlled:** heating/cooling setpoints determined once per simulated hour by a local LLM (Llama 3.2) based on live outdoor temperature, zone temperature, and PMV comfort index, validated against ASHRAE 55 deadband and safety bounds before being applied.
- Electricity totals are computed by summing all hourly HVAC-relevant electricity submeters (fans, cooling, heating, interior lighting, interior equipment) across all 672 fifteen-minute timesteps in the evaluation week.
- Comfort is measured via the Fanger PMV model for the building's core occupied zone, sampled at every 15-minute zone timestep.

## Visual Comparison

![Results comparison chart](results_comparison_chart.png)

## Reproducing These Results

```powershell
python run_baseline.py
python llm_control.py
python dashboard.py
python make_chart.py
```

`dashboard.py` prints the full breakdown (row counts, PMV statistics, electricity totals per run) to the console; the table above reflects its output for the 07/14-07/20 evaluation week.
```

# tools.py

```
class ToolServer:
    """
    Wraps read/write access to the EnergyPlus simulation as named 'tools',
    the same way an LLM would call them. The EnergyPlus callback calls
    these methods instead of touching actuators directly.
    """

    def __init__(self):
        # live sensor values, updated by the EnergyPlus callback each timestep
        self.zone_temp = None
        self.zone_pmv = None
        self.outdoor_temp = None
        self.sim_time = None

        # control values, read by the EnergyPlus callback each timestep
        self.heating_setpoint = 21.0
        self.cooling_setpoint = 24.0

    def get_zone_conditions(self):
        return {
            "zone_temperature_c": round(self.zone_temp, 2) if self.zone_temp is not None else None,
            "zone_pmv": round(self.zone_pmv, 2) if self.zone_pmv is not None else None,
        }

    def get_site_conditions(self):
        return {
            "outdoor_temperature_c": round(self.outdoor_temp, 2) if self.outdoor_temp is not None else None,
            "sim_time": self.sim_time,
        }

    def set_hvac_setpoints(self, heating_c, cooling_c):
        DEADBAND_MIN = 1.1
        if cooling_c < heating_c + DEADBAND_MIN:
            return {
                "accepted": False,
                "error": f"cooling_c ({cooling_c}) must be >= heating_c ({heating_c}) + {DEADBAND_MIN}",
            }
        self.heating_setpoint = heating_c
        self.cooling_setpoint = cooling_c
        return {"accepted": True, "heating_c": heating_c, "cooling_c": cooling_c}
```



# llm\_decide.py

```
import json
import urllib.request

def ask_llm_for_setpoints(outdoor_temp, zone_temp, zone_pmv, model="llama3.2"):
    """
    Sends current conditions to a local Ollama model and asks it to decide
    heating/cooling setpoints. Returns (heating_c, cooling_c) as floats,
    or None if anything goes wrong (caller should fall back to a safe default).
    """
    prompt = (
        f"You control a building's HVAC. Current conditions: "
        f"outdoor temperature = {outdoor_temp:.1f}C, zone temperature = {zone_temp:.1f}C, "
        f"zone comfort PMV = {zone_pmv:.2f} (target range: -0.5 to 0.5). "
        f"Reply with ONLY two numbers separated by a comma: heating_setpoint,cooling_setpoint "
        f"in Celsius. heating must be between 15 and 22. cooling must be between 22 and 28. "
        f"cooling must be at least 1.1 higher than heating. Example reply: 21.0,24.0"
    )

    payload = {
        "model": model,
        "prompt": prompt,
        "stream": False,
    }

    try:
        req = urllib.request.Request(
            "http://localhost:11434/api/generate",
            data=json.dumps(payload).encode(),
            headers={"Content-Type": "application/json"},
            method="POST",
        )
        with urllib.request.urlopen(req, timeout=15) as resp:
            data = json.loads(resp.read().decode())
            text = data["response"].strip()
            heat_str, cool_str = text.split(",")
            return float(heat_str.strip()), float(cool_str.strip())
    except Exception as e:
        print(f"LLM call failed: {e}")
        return None

if __name__ == "__main__":
    result = ask_llm_for_setpoints(outdoor_temp=30.0, zone_temp=25.0, zone_pmv=0.1, model="llama3.2")
    print("LLM returned:", result)
```

# llm\_control.py

```
import sys

EPLUS_ROOT = r"C:\EnergyPlusV26-1-0"
sys.path.insert(0, EPLUS_ROOT)

from pyenergyplus.api import EnergyPlusAPI
from tools import ToolServer
from llm_decide import ask_llm_for_setpoints

api = EnergyPlusAPI()
state = api.state_manager.new_state()
tools = ToolServer()

handles = {"temp": -1, "pmv": -1, "outdoor": -1, "heat_act": -1, "cool_act": -1}
call_count = [0]
setpoint_log = []

# throttle LLM calls: only ask the LLM once every N zone timesteps, since
# each call takes real wall-clock seconds and we don't want to slow the
# simulation to a crawl or hammer Ollama. At 15-min timesteps, N=4 means
# "ask once per simulated hour" - and we HOLD the last decision in between.
LLM_CALL_EVERY_N_STEPS = 4

def my_callback(state):
    ex = api.exchange

    if ex.warmup_flag(state):
        return

    if handles["temp"] == -1:
        handles["temp"] = ex.get_variable_handle(state, "Zone Mean Air Temperature", "Core_bottom")
        handles["pmv"] = ex.get_variable_handle(state, "Zone Thermal Comfort Fanger Model PMV", "Core_bottom
People")
        handles["outdoor"] = ex.get_variable_handle(state, "Site Outdoor Air Drybulb Temperature",
"Environment")
        handles["heat_act"] = ex.get_actuator_handle(state, "Schedule:Compact", "Schedule Value",
"HTGSETP_SCH")
        handles["cool_act"] = ex.get_actuator_handle(state, "Schedule:Compact", "Schedule Value",
"CLGSETP_SCH")
        if -1 in handles.values():
            print("ERROR: could not resolve one or more handles")
            return

    # READ
    tools.zone_temp = ex.get_variable_value(state, handles["temp"])
    tools.zone_pmv = ex.get_variable_value(state, handles["pmv"])
    tools.outdoor_temp = ex.get_variable_value(state, handles["outdoor"])
    tools.sim_time = f"{ex.month(state):02d}/{ex.day_of_month(state):02d} {ex.hour(state):02d}:00"

    call_count[0] += 1

    # DECIDE (only every N steps - LLM decision is HELD between calls)
    if call_count[0] % LLM_CALL_EVERY_N_STEPS == 0:
        result = ask_llm_for_setpoints(
            outdoor_temp=tools.outdoor_temp,
            zone_temp=tools.zone_temp,
            zone_pmv=tools.zone_pmv,
        )
        if result is not None:
            heat_c, cool_c = result
            tool_result = tools.set_hvac_setpoints(heating_c=heat_c, cooling_c=cool_c)
            if not tool_result["accepted"]:
                print(f" [LLM proposal REJECTED by validator: {tool_result['error']}] -> holding previous
setpoints")
            else:
```

```

        print(" [LLM call failed -> holding previous setpoints]")

# WRITE (every timestep, holding the last accepted decision)
ex.set_actuator_value(state, handles["heat_act"], tools.heating_setpoint)
ex.set_actuator_value(state, handles["cool_act"], tools.cooling_setpoint)

if call_count[0] % LLM_CALL_EVERY_N_STEPS != 0:
    return

zc = tools.get_zone_conditions()
sc = tools.get_site_conditions()
print(f"[{sc['sim_time']}] outdoor={sc['outdoor_temperature_c']}C "
      f"zone_temp={zc['zone_temperature_c']}C pmv={zc['zone_pmv']} "
      f"-> LLM setpoints=({tools.heating_setpoint}, {tools.cooling_setpoint})")

setpoint_log.append({
    "sim_time": sc["sim_time"],
    "month": ex.month(state),
    "day": ex.day_of_month(state),
    "hour": ex.hour(state),
    "heating_setpoint": tools.heating_setpoint,
    "cooling_setpoint": tools.cooling_setpoint,
})

api.runtime.callback_begin_zone_timestep_after_init_heat_balance(state, my_callback)

args = [
    "-w", r"weather\chicago.epw",
    "-d", r"results_llm_control",
    "-r", r"idf\baseline.idf",
]

api.runtime.run_energyplus(state, args)

import csv
with open("llm_setpoint_log.csv", "w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["sim_time", "month", "day", "hour", "heating_setpoint",
    "cooling_setpoint"])
    writer.writeheader()
    writer.writerows(setpoint_log)
print(f"Saved {len(setpoint_log)} setpoint decisions to llm_setpoint_log.csv")

```

## run\_baseline.py

```
#!/usr/bin/env python3
"""
Runs the building exactly as authored: DOE Reference Medium Office fixed
heating/cooling setpoint schedules (21C occupied heating / 24C occupied
cooling, night/weekend setback) -- the traditional rigid, rule-based BMS
described in the problem statement. No AI in the loop.
"""

import argparse
import os
import subprocess
import sys

ROOT = os.path.dirname(os.path.abspath(__file__))
EPLUS_ROOT = os.environ.get("EPLUS_ROOT", r"C:\EnergyPlusV26-1-0")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--idf", default=os.path.join(ROOT, "idf", "baseline.idf"))
    ap.add_argument("--weather", default=os.path.join(ROOT, "weather", "chicago.epw"))
    ap.add_argument("--out", default=os.path.join(ROOT, "results"))
    args = ap.parse_args()

    os.makedirs(args.out, exist_ok=True)
    eplus_bin = os.path.join(EPLUS_ROOT, "energyplus.exe")
    cmd = [eplus_bin, "-w", args.weather, "-d", args.out, "-r", args.idf]
    print("Running baseline (traditional rule-based BMS):", " ".join(cmd))
    result = subprocess.run(cmd, capture_output=True, text=True)
    print(result.stdout[-2000:])
    if result.returncode != 0:
        print(result.stderr, file=sys.stderr)
        sys.exit(result.returncode)
    print(f"Baseline complete. Outputs in {args.out}")

if __name__ == "__main__":
    main()
```

## dashboard.py

```
import csv

def load_csv(path):
    with open(path, newline="") as f:
        reader = csv.reader(f)
        header = next(reader)
        rows = list(reader)
    return header, rows

def col_index(header, exact_name):
    for i, h in enumerate(header):
        if h.strip() == exact_name:
            return i
    raise ValueError(f"Column '{exact_name}' not found. Available: {header}")

def parse_datetime(d):
    d = d.strip()
    date_part, time_part = d.split(" ")
    month, day = date_part.split("/")
    hh, mm, ss = time_part.split(":")
    return int(month), int(day), int(hh), int(mm), int(ss)

def week_hourly_rows(rows):
    """Filter to hourly rows (mm=0, ss=0) within 07/14-07/20."""
    out = []
    for r in rows:
        try:
            m, day, hh, mm, ss = parse_datetime(r[0])
        except Exception:
            continue
        if m == 7 and 14 <= day <= 20 and mm == 0 and ss == 0:
            out.append(r)
    return out

def week_quarter_hour_rows(rows):
    """Filter to true 15-min zone-timestep rows within 07/14-07/20."""
    out = []
    for r in rows:
        try:
            m, day, hh, mm, ss = parse_datetime(r[0])
        except Exception:
            continue
        if m == 7 and 14 <= day <= 20 and mm in (0, 15, 30, 45) and ss == 0:
            out.append(r)
    return out

ELECTRICITY_SUBMETERS = [
    "Fans:Electricity [J](Hourly)",
    "Cooling:Electricity [J](Hourly)",
    "Heating:Electricity [J](Hourly)",
    "InteriorLights:Electricity [J](Hourly)",
    "InteriorEquipment:Electricity [J](Hourly)",
]

def total_electricity_kwh(mtr_path):
    header, rows = load_csv(mtr_path)
    week_rows = week_hourly_rows(rows)
    indices = [col_index(header, name) for name in ELECTRICITY_SUBMETERS]

    total_j = 0.0
    rows_used = 0
    for r in week_rows:
        if len(r) <= max(indices):
            continue
        row_total = sum(float(r[i]) for i in indices)
```

```

    total_j += row_total
    rows_used += 1

print(f" (electricity: summed {len(indices)} submeters across {rows_used} hourly rows)")
return total_j / 3.6e6 # J -> kWh

def comfort_stats(out_path):
    header, rows = load_csv(out_path)
    week_rows = week_quarter_hour_rows(rows)
    pmv_idx = None
    for i, h in enumerate(header):
        if "CORE_BOTTOM PEOPLE:Zone Thermal Comfort Fanger Model PMV" in h:
            pmv_idx = i
            break
    if pmv_idx is None:
        raise ValueError("PMV column not found")

    pmv_vals = [float(r[pmv_idx]) for r in week_rows if len(r) > pmv_idx and r[pmv_idx].strip() != ""]
    avg_pmv = sum(pmv_vals) / len(pmv_vals)
    max_abs_pmv = max(abs(v) for v in pmv_vals)
    violations = sum(1 for v in pmv_vals if abs(v) > 0.5)
    violation_pct = 100 * violations / len(pmv_vals)
    return avg_pmv, max_abs_pmv, violation_pct, len(pmv_vals)

def analyze(out_path, mtr_path, label):
    print(f"\n=== {label} ===")
    kwh = total_electricity_kwh(mtr_path)
    avg_pmv, max_pmv, viol_pct, n = comfort_stats(out_path)
    print(f"Total electricity:    {kwh:.1f} kWh")
    print(f"PMV rows analyzed:      {n}")
    print(f"Average PMV:           {avg_pmv:.3f}")
    print(f"Max |PMV| deviation:    {max_pmv:.3f}")
    print(f"Comfort violations:      {viol_pct:.1f}% of timesteps with |PMV| > 0.5")
    return kwh, avg_pmv, viol_pct

if __name__ == "__main__":
    baseline_kwh, baseline_pmv, baseline_viol = analyze(
        r"results\eplusout.csv", r"results\eplusmtr.csv", "BASELINE (fixed schedule)"
    )
    ai_kwh, ai_pmv, ai_viol = analyze(
        r"results_llm_control\eplusout.csv", r"results_llm_control\eplusmtr.csv", "AI-CONTROLLED (LLM)"
    )

    savings_pct = 100 * (baseline_kwh - ai_kwh) / baseline_kwh

    print("\n=== COMPARISON ===")
    print(f"Energy: baseline={baseline_kwh:.1f} kWh, AI={ai_kwh:.1f} kWh -> {savings_pct:+.1f}% change")
    print(f"Comfort violations: baseline={baseline_viol:.1f}%, AI={ai_viol:.1f}%")

```

## make\_chart.py

```
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

baseline_kwh = 16142.2
ai_kwh = 15198.7
baseline_viol_pct = 40.0
ai_viol_pct = 23.1

GRAY = "#9CA3AF"
BLUE = "#2563EB"
GREEN = "#16A34A"
TEXT_DARK = "#1F2937"

plt.rcParams["font.family"] = "sans-serif"

fig, axes = plt.subplots(1, 2, figsize=(11, 5.6))
fig.patch.set_facecolor("white")

def style_axis(ax):
    ax.spines["top"].set_visible(False)
    ax.spines["right"].set_visible(False)
    ax.spines["left"].set_visible(False)
    ax.yaxis.grid(True, color="#E5E7EB", linewidth=0.8, zorder=0)
    ax.set_axisbelow(True)
    ax.tick_params(axis="both", length=0, labelsize=10, colors=TEXT_DARK)

def drawBars(ax, values, labels, value_fmt, ylabel, title, delta_text):
    bars = ax.bar(labels, values, color=[GRAY, BLUE], width=0.55, zorder=3)

    # value labels directly above each bar
    for bar, v in zip(bars, values):
        ax.annotate(value_fmt(v), xy=(bar.get_x() + bar.get_width() / 2, bar.get_height()),
                    xytext=(0, 6), textcoords="offset points", ha="center",
                    fontsize=12, fontweight="bold", color=TEXT_DARK)

    ax.set_ylabel(ylabel, fontsize=10.5, color=TEXT_DARK, labelpad=8)
    ax.set_title(title, fontsize=13, fontweight="bold", color=TEXT_DARK, pad=16)

    # give generous headroom above the tallest bar for the connector + label
    ymax = max(values) * 1.32
    ax.set_ylim(0, ymax)

    # connector line + delta label, drawn INSIDE the plot area (data coords),
    # so it can never collide with the title above the axes
    x0 = bars[0].get_x() + bars[0].get_width() / 2
    x1 = bars[1].get_x() + bars[1].get_width() / 2
    connector_y = max(values) * 1.12
    ax.plot([x0, x1], [connector_y, connector_y], color=GREEN, linewidth=1.4, zorder=4)
    ax.plot([x0, x0], [connector_y - ymax * 0.015, connector_y], color=GREEN, linewidth=1.4, zorder=4)
    ax.plot([x1, x1], [connector_y - ymax * 0.015, connector_y], color=GREEN, linewidth=1.4, zorder=4)
    ax.text((x0 + x1) / 2, connector_y + ymax * 0.025, delta_text,
            ha="center", va="bottom", fontsize=13, fontweight="bold", color=GREEN, zorder=5)

    style_axis(ax)

savings_pct = 100 * (baseline_kwh - ai_kwh) / baseline_kwh
viol_reduction = baseline_viol_pct - ai_viol_pct

drawBars(
    axes[0], [baseline_kwh, ai_kwh],
    ["Baseline\n(fixed schedule)", "AI-Controlled\n(LLM)"],
    lambda v: f"{v:,.0f} kWh",
    "Total Electricity (kWh)", "Weekly Energy Consumption",
    f"\u2193 {savings_pct:.1f}%",
```

```

)

draw_bars(
    axes[1], [baseline_viol_pct, ai_viol_pct],
    ["Baseline\n(fixed schedule)", "AI-Controlled\n(LLM)"],
    lambda v: f"{v:.1f}%",
    "% of Timesteps with |PMV| > 0.5", "Thermal Comfort Violations",
    f"\u2193 {viol_reduction:.1f} pts",
)

fig.suptitle("Eco-Loop: Baseline vs. AI-Controlled HVAC",
             fontsize=15, fontweight="bold", color=TEXT_DARK, y=1.02)
fig.text(0.5, 0.955, "07/14 \u2013 07/20 \u00b7 Chicago Peak Summer Week \u00b7 DOE Medium Office",
         ha="center", fontsize=10.5, color="#6B7280")

plt.tight_layout(rect=[0, 0, 1, 0.93])
plt.savefig("results_comparison_chart.png", dpi=220, bbox_inches="tight", facecolor="white")
print("Saved: results_comparison_chart.png")

```



## export\_modified\_idf.py

```
"""
export_modified_idf.py
-----
Deliverable #2 support: exports a "modified" version of the baseline IDF
where the original fixed HTGSETP_SCH / CLGSETP_SCH schedules are replaced
with the actual hour-by-hour setpoint sequence the LLM produced during the
AI-controlled run (from llm_setpoint_log.csv). This lets anyone re-run the
exact AI-driven setpoint sequence through EnergyPlus without needing the
live LLM/Ollama loop.
"""

import csv
import sys

EPLUS_ROOT = r"C:\EnergyPlusV26-1-0"
sys.path.insert(0, EPLUS_ROOT)

from eppy.modeleditor import IDF

IDF.setiddname(EPLUS_ROOT + r"\Energy+.idd")

# --- load the LLM's actual applied setpoint sequence ---
with open("llm_setpoint_log.csv", newline="") as f:
    reader = csv.DictReader(f)
    log_rows = list(reader)

print(f"Loaded {len(log_rows)} logged setpoint decisions")

# keep only the real evaluation week (07/14-07/20), in case earlier
# sizing-day rows got logged too
week_rows = [r for r in log_rows if r["month"] == "7" and 14 <= int(r["day"]) <= 20]
print(f"{len(week_rows)} rows are within the 07/14-07/20 evaluation week")

# --- build Schedule:Compact field lists for heating and cooling ---
# Schedule:Compact format: Through: date, For: days, Until: HH:MM, value, ...
# We build one "Until" entry per logged hour, grouped by day.

def build_schedule_fields(week_rows, key):
    fields = []
    days_seen = sorted(set((int(r["month"]), int(r["day"])) for r in week_rows))
    for month, day in days_seen:
        day_rows = [r for r in week_rows if int(r["month"]) == month and int(r["day"]) == day]
        day_rows.sort(key=lambda r: int(r["hour"]))
        fields.append(f"Through: {month:02d}/{day:02d}")
        fields.append("For: AllDays")
        for r in day_rows:
            hour = int(r["hour"])
            end_hour = hour + 1
            fields.append(f"Until: {end_hour:02d}:00")
            fields.append(r[key])
    return fields

heat_fields = build_schedule_fields(week_rows, "heating_setpoint")
cool_fields = build_schedule_fields(week_rows, "cooling_setpoint")

# --- edit the IDF ---
idf = IDF("idf/baseline.idf")

for sched_name, fields, out_name in [
    ("HTGSETP_SCH", heat_fields, "AI_HTGSETP_SCH"),
    ("CLGSETP_SCH", cool_fields, "AI_CLGSETP_SCH"),
]:
    old = [s for s in idf.idfobjects["SCHEDULE:COMPACT"] if s.Name == sched_name][0]
    sched_type = old.Schedule_Type_Limits_Name
    idf.removeidfobject(old)
```

```
new_sched = idf.newidfobject("SCHEDULE:COMPACT", Name=out_name,
                             Schedule_Type_Limits_Name=sched_type)
# bypass eppy's Field_N attribute mechanism (breaks on dynamic extension);
# directly extend the underlying field-value list instead
new_sched.obj.extend(fields)

# repoint the dual setpoint thermostats to the renamed AI schedules
for dsp in idf.idfobjects["THERMOSTATSETPOINT:DUALSETPOINT"]:
    dsp.Heating_Setpoint_Temperature_Schedule_Name = "AI_HTGSETP_SCH"
    dsp.Cooling_Setpoint_Temperature_Schedule_Name = "AI_CLGSETP_SCH"

idf.saveas("idf/ai_controlled.idf")
print("Saved idf/ai_controlled.idf")
```

# llm\_setpoint\_log.csv

```
sim_time,month,day,hour,heating_setpoint,cooling_setpoint
01/21 00:00,1,21,0,18.5,26.2
01/21 01:00,1,21,1,18.5,26.0
01/21 02:00,1,21,2,17.8,25.9
01/21 03:00,1,21,3,18.0,26.0
01/21 04:00,1,21,4,18.5,25.5
01/21 05:00,1,21,5,19.5,26.3
01/21 06:00,1,21,6,20.5,25.9
01/21 07:00,1,21,7,18.0,26.0
01/21 08:00,1,21,8,18.5,26.2
01/21 09:00,1,21,9,18.0,26.5
01/21 10:00,1,21,10,19.0,26.0
01/21 11:00,1,21,11,21.5,25.2
01/21 12:00,1,21,12,18.0,25.5
01/21 13:00,1,21,13,18.5,25.5
01/21 14:00,1,21,14,18.5,26.2
01/21 15:00,1,21,15,18.0,25.5
01/21 16:00,1,21,16,19.8,26.1
01/21 17:00,1,21,17,18.2,26.5
01/21 18:00,1,21,18,19.5,26.8
01/21 19:00,1,21,19,19.5,25.0
01/21 20:00,1,21,20,18.0,26.0
01/21 21:00,1,21,21,19.0,25.5
01/21 22:00,1,21,22,18.5,25.5
01/21 23:00,1,21,23,20.5,26.0
07/21 00:00,7,21,0,18.0,26.0
07/21 01:00,7,21,1,16.7,25.5
07/21 02:00,7,21,2,18.2,26.5
07/21 03:00,7,21,3,19.4,25.6
07/21 04:00,7,21,4,18.5,26.3
07/21 05:00,7,21,5,18.5,26.5
07/21 06:00,7,21,6,19.0,25.5
07/21 07:00,7,21,7,18.5,26.6
07/21 08:00,7,21,8,18.5,26.8
07/21 09:00,7,21,9,21.0,24.5
07/21 10:00,7,21,10,16.5,26.6
07/21 11:00,7,21,11,19.5,25.6
07/21 12:00,7,21,12,20.5,26.8
07/21 13:00,7,21,13,19.5,26.3
07/21 14:00,7,21,14,19.5,26.5
07/21 15:00,7,21,15,18.5,26.5
07/21 16:00,7,21,16,19.5,25.1
07/21 17:00,7,21,17,20.5,26.7
07/21 18:00,7,21,18,19.4,25.6
07/21 19:00,7,21,19,16.5,25.3
07/21 20:00,7,21,20,19.5,26.2
07/21 21:00,7,21,21,19.5,26.8
07/21 22:00,7,21,22,18.5,26.5
07/21 23:00,7,21,23,20.5,25.8
01/21 00:00,1,21,0,17.6,25.2
01/21 01:00,1,21,1,17.6,26.5
01/21 02:00,1,21,2,19.5,26.2
01/21 03:00,1,21,3,18.0,25.5
01/21 04:00,1,21,4,17.5,25.6
01/21 05:00,1,21,5,17.5,25.6
01/21 06:00,1,21,6,18.8,25.4
01/21 07:00,1,21,7,19.4,26.6
01/21 08:00,1,21,8,20.5,26.3
01/21 09:00,1,21,9,18.5,25.5
01/21 10:00,1,21,10,20.5,26.2
01/21 11:00,1,21,11,19.5,26.7
01/21 12:00,1,21,12,18.8,26.2
01/21 13:00,1,21,13,17.5,25.5
01/21 14:00,1,21,14,18.5,26.2
01/21 15:00,1,21,15,19.5,26.3
```

01/21 16:00,1,21,16,20.8,25.5  
01/21 17:00,1,21,17,18.8,26.9  
01/21 18:00,1,21,18,18.8,25.6  
01/21 19:00,1,21,19,20.9,26.7  
01/21 20:00,1,21,20,18.5,26.2  
01/21 21:00,1,21,21,19.9,26.1  
01/21 22:00,1,21,22,18.5,25.5  
01/21 23:00,1,21,23,19.8,25.3  
07/21 00:00,7,21,0,18.5,26.7  
07/21 01:00,7,21,1,18.0,25.0  
07/21 02:00,7,21,2,20.5,26.7  
07/21 03:00,7,21,3,18.5,25.1  
07/21 04:00,7,21,4,19.5,26.2  
07/21 05:00,7,21,5,18.5,26.2  
07/21 06:00,7,21,6,19.5,26.7  
07/21 07:00,7,21,7,19.9,26.4  
07/21 08:00,7,21,8,20.5,26.8  
07/21 09:00,7,21,9,20.5,25.5  
07/21 10:00,7,21,10,18.5,26.9  
07/21 11:00,7,21,11,20.5,26.2  
07/21 12:00,7,21,12,19.5,25.7  
07/21 13:00,7,21,13,18.2,26.5  
07/21 14:00,7,21,14,18.5,25.6  
07/21 15:00,7,21,15,18.0,26.0  
07/21 16:00,7,21,16,18.0,26.5  
07/21 17:00,7,21,17,19.5,25.6  
07/21 18:00,7,21,18,18.0,26.0  
07/21 19:00,7,21,19,19.5,26.9  
07/21 20:00,7,21,20,19.5,25.8  
07/21 21:00,7,21,21,19.5,25.8  
07/21 22:00,7,21,22,19.5,26.2  
07/21 23:00,7,21,23,20.5,25.5  
07/14 00:00,7,14,0,19.5,26.4  
07/14 01:00,7,14,1,20.5,26.8  
07/14 02:00,7,14,2,20.5,26.5  
07/14 03:00,7,14,3,19.5,26.5  
07/14 04:00,7,14,4,20.9,25.4  
07/14 05:00,7,14,5,18.6,26.3  
07/14 06:00,7,14,6,19.5,26.8  
07/14 07:00,7,14,7,19.5,26.8  
07/14 08:00,7,14,8,20.5,26.5  
07/14 09:00,7,14,9,18.5,26.2  
07/14 10:00,7,14,10,20.5,26.6  
07/14 11:00,7,14,11,20.5,26.1  
07/14 12:00,7,14,12,19.5,26.8  
07/14 13:00,7,14,13,19.5,25.8  
07/14 14:00,7,14,14,19.5,25.5  
07/14 15:00,7,14,15,18.8,26.2  
07/14 16:00,7,14,16,19.5,25.3  
07/14 17:00,7,14,17,21.5,26.3  
07/14 18:00,7,14,18,18.5,25.3  
07/14 19:00,7,14,19,17.9,25.3  
07/14 20:00,7,14,20,20.5,25.5  
07/14 21:00,7,14,21,18.5,26.7  
07/14 22:00,7,14,22,19.5,25.3  
07/14 23:00,7,14,23,20.5,26.1  
07/15 00:00,7,15,0,18.9,25.1  
07/15 01:00,7,15,1,18.9,25.1  
07/15 02:00,7,15,2,18.2,26.4  
07/15 03:00,7,15,3,19.5,26.5  
07/15 04:00,7,15,4,20.5,27.1  
07/15 05:00,7,15,5,18.5,26.2  
07/15 06:00,7,15,6,20.5,25.5  
07/15 07:00,7,15,7,20.5,25.7  
07/15 08:00,7,15,8,19.5,26.1  
07/15 09:00,7,15,9,18.5,25.5  
07/15 10:00,7,15,10,18.0,25.0  
07/15 11:00,7,15,11,18.5,26.6

07/15 12:00,7,15,12,20.5,26.5  
07/15 13:00,7,15,13,20.5,27.1  
07/15 14:00,7,15,14,19.5,26.3  
07/15 15:00,7,15,15,20.5,27.4  
07/15 16:00,7,15,16,18.5,25.1  
07/15 17:00,7,15,17,19.8,26.1  
07/15 18:00,7,15,18,18.5,25.2  
07/15 19:00,7,15,19,19.5,26.6  
07/15 20:00,7,15,20,19.5,25.5  
07/15 21:00,7,15,21,19.4,26.9  
07/15 22:00,7,15,22,20.0,25.5  
07/15 23:00,7,15,23,19.8,25.3  
07/16 00:00,7,16,0,18.5,26.5  
07/16 01:00,7,16,1,18.0,26.0  
07/16 02:00,7,16,2,19.5,26.3  
07/16 03:00,7,16,3,18.5,26.9  
07/16 04:00,7,16,4,20.5,25.2  
07/16 05:00,7,16,5,18.2,26.6  
07/16 06:00,7,16,6,20.5,27.1  
07/16 07:00,7,16,7,19.4,25.9  
07/16 08:00,7,16,8,20.5,25.5  
07/16 09:00,7,16,9,19.8,25.2  
07/16 10:00,7,16,10,19.5,26.7  
07/16 11:00,7,16,11,18.5,25.7  
07/16 12:00,7,16,12,19.5,25.9  
07/16 13:00,7,16,13,18.5,25.7  
07/16 14:00,7,16,14,18.5,25.0  
07/16 15:00,7,16,15,17.5,26.1  
07/16 16:00,7,16,16,20.5,26.2  
07/16 17:00,7,16,17,19.4,25.6  
07/16 18:00,7,16,18,16.6,26.7  
07/16 19:00,7,16,19,20.5,26.2  
07/16 20:00,7,16,20,21.5,26.3  
07/16 21:00,7,16,21,18.5,25.3  
07/16 22:00,7,16,22,20.5,25.3  
07/16 23:00,7,16,23,18.5,26.1  
07/17 00:00,7,17,0,19.2,26.5  
07/17 01:00,7,17,1,20.5,26.7  
07/17 02:00,7,17,2,19.8,25.5  
07/17 03:00,7,17,3,20.5,26.3  
07/17 04:00,7,17,4,19.5,26.5  
07/17 05:00,7,17,5,20.5,26.7  
07/17 06:00,7,17,6,18.5,25.7  
07/17 07:00,7,17,7,19.5,26.6  
07/17 08:00,7,17,8,18.2,26.5  
07/17 09:00,7,17,9,20.5,27.1  
07/17 10:00,7,17,10,20.5,26.2  
07/17 11:00,7,17,11,18.5,25.5  
07/17 12:00,7,17,12,19.5,26.6  
07/17 13:00,7,17,13,18.8,25.5  
07/17 14:00,7,17,14,19.5,26.5  
07/17 15:00,7,17,15,19.5,26.9  
07/17 16:00,7,17,16,18.5,26.2  
07/17 17:00,7,17,17,18.2,26.5  
07/17 18:00,7,17,18,20.5,26.2  
07/17 19:00,7,17,19,20.8,26.2  
07/17 20:00,7,17,20,17.5,25.8  
07/17 21:00,7,17,21,18.0,25.8  
07/17 22:00,7,17,22,19.5,25.2  
07/17 23:00,7,17,23,18.5,25.6  
07/18 00:00,7,18,0,20.5,26.4  
07/18 01:00,7,18,1,19.5,26.7  
07/18 02:00,7,18,2,18.5,26.3  
07/18 03:00,7,18,3,18.0,25.5  
07/18 04:00,7,18,4,20.5,26.1  
07/18 05:00,7,18,5,19.5,26.2  
07/18 06:00,7,18,6,19.5,25.9  
07/18 07:00,7,18,7,20.5,26.7

07/18 08:00,7,18,8,20.5,26.2  
07/18 09:00,7,18,9,20.5,26.7  
07/18 10:00,7,18,10,19.5,25.6  
07/18 11:00,7,18,11,20.8,25.3  
07/18 12:00,7,18,12,18.5,26.2  
07/18 13:00,7,18,13,18.8,25.6  
07/18 14:00,7,18,14,17.8,26.5  
07/18 15:00,7,18,15,19.5,25.9  
07/18 16:00,7,18,16,20.5,26.7  
07/18 17:00,7,18,17,19.5,26.2  
07/18 18:00,7,18,18,19.5,26.2  
07/18 19:00,7,18,19,20.5,26.2  
07/18 20:00,7,18,20,18.5,25.5  
07/18 21:00,7,18,21,19.5,25.8  
07/18 22:00,7,18,22,20.5,26.2  
07/18 23:00,7,18,23,19.5,26.7  
07/19 00:00,7,19,0,21.0,26.5  
07/19 01:00,7,19,1,20.5,27.1  
07/19 02:00,7,19,2,20.5,25.8  
07/19 03:00,7,19,3,17.9,25.5  
07/19 04:00,7,19,4,18.5,26.3  
07/19 05:00,7,19,5,19.5,26.1  
07/19 06:00,7,19,6,19.5,25.8  
07/19 07:00,7,19,7,20.5,26.2  
07/19 08:00,7,19,8,19.4,26.8  
07/19 09:00,7,19,9,20.0,25.5  
07/19 10:00,7,19,10,19.8,25.5  
07/19 11:00,7,19,11,19.5,25.2  
07/19 12:00,7,19,12,19.5,25.9  
07/19 13:00,7,19,13,20.5,26.3  
07/19 14:00,7,19,14,19.4,26.2  
07/19 15:00,7,19,15,18.4,25.6  
07/19 16:00,7,19,16,18.2,26.5  
07/19 17:00,7,19,17,19.5,26.3  
07/19 18:00,7,19,18,18.5,26.5  
07/19 19:00,7,19,19,19.7,26.1  
07/19 20:00,7,19,20,18.5,25.5  
07/19 21:00,7,19,21,17.5,26.5  
07/19 22:00,7,19,22,18.5,26.2  
07/19 23:00,7,19,23,19.5,26.8  
07/20 00:00,7,20,0,18.8,26.2  
07/20 01:00,7,20,1,18.5,26.2  
07/20 02:00,7,20,2,19.5,26.1  
07/20 03:00,7,20,3,19.5,26.2  
07/20 04:00,7,20,4,18.5,26.2  
07/20 05:00,7,20,5,19.8,25.9  
07/20 06:00,7,20,6,19.5,25.2  
07/20 07:00,7,20,7,18.5,26.6  
07/20 08:00,7,20,8,19.5,26.7  
07/20 09:00,7,20,9,20.5,26.2  
07/20 10:00,7,20,10,18.5,26.5  
07/20 11:00,7,20,11,19.9,26.8  
07/20 12:00,7,20,12,19.5,26.6  
07/20 13:00,7,20,13,19.4,26.2  
07/20 14:00,7,20,14,18.0,25.5  
07/20 15:00,7,20,15,18.5,26.7  
07/20 16:00,7,20,16,18.2,25.8  
07/20 17:00,7,20,17,20.5,26.2  
07/20 18:00,7,20,18,20.5,26.2  
07/20 19:00,7,20,19,18.5,25.2  
07/20 20:00,7,20,20,19.5,25.7  
07/20 21:00,7,20,21,20.6,26.9  
07/20 22:00,7,20,22,18.2,26.5  
07/20 23:00,7,20,23,18.5,26.9

# idf/baseline.idf and idf/ai\_controlled.idf (excerpt)

Both full building model files (baseline.idf and ai\_controlled.idf) are included in the GitHub repository under idf/. The full baseline file is ~8,300 lines (standard EnergyPlus building geometry, materials, and HVAC definitions) and is not reproduced here in full. Below is the AI-generated setpoint schedule embedded in ai\_controlled.idf, showing the actual hour-by-hour heating/cooling setpoints the LLM produced during the evaluation week, replacing the original fixed HTGSETP\_SCH / CLGSETP\_SCH schedules.

```
AI_HTGSETP_SCH,      !- Name
Temperature,         !- Schedule Type Limits Name
Through: 07/14,      !- Field 1
For: AllDays,        !- Field 2
Until: 01:00,        !- Field 3
19.8,                !- Field 4
Until: 02:00,        !- Field 5
20.5,                !- Field 6
Until: 03:00,        !- Field 7
20.5,                !- Field 8
Until: 04:00,        !- Field 9
19.5,                !- Field 10
Until: 05:00,        !- Field 11
19.5,                !- Field 12
Until: 06:00,        !- Field 13
17.6,                !- Field 14
Until: 07:00,        !- Field 15
19.5,                !- Field 16
Until: 08:00,        !- Field 17
18.5,                !- Field 18
Until: 09:00,        !- Field 19
19.5,                !- Field 20
Until: 10:00,        !- Field 21
18.9,                !- Field 22
Until: 11:00,        !- Field 23
18.5,                !- Field 24
Until: 12:00,        !- Field 25
19.5,                !- Field 26
Until: 13:00,        !- Field 27
19.5,                !- Field 28
Until: 14:00,        !- Field 29
20.5,                !- Field 30
Until: 15:00,        !- Field 31
20.5,                !- Field 32
Until: 16:00,        !- Field 33
18.0,                !- Field 34
Until: 17:00,        !- Field 35
20.5,                !- Field 36
Until: 18:00,        !- Field 37
18.0,                !- Field 38
```

Eco-Loop: Baseline vs. AI-Controlled HVAC

07/14 - 07/20 · Chicago Peak Summer Week · DOE Medium Office

